SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA1419 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool *(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 3 **Total Marks:50**

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

*I **E.Ragul / 192571032** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
E.Ragul

Annexure A

ERROR ANALYSIS TASK

Q1. The following program is intended to perform simple arithmetic operations, function calls, array access, and looping constructs. However, the code contains multiple errors introduced at different phases of compilation, including lexical, syntax, semantic, logical, and runtime errors.

Given Program

```
#include <stdio.h>

int sumArray(int arr[], int n) {
   int i, sum = 0;
   for(i = 0; i <= n; i++) {
      sum = sum + arr[i];
   }
   return sum;
}

int main() {
   int arr[5] = {1,2,3,4,5}
   int total;

   total = sumArray(arr);

   if(total == 15) {
      printf("Correct Sum\n")
   }

   int a = 10, b = 0;
   int c = a / b;

   int *p;
   *p = 20;

   return 0;
}
```

Tasks:

(a) Error Identification (Analysis Level)

1. Identify all the errors present in the program. For each error:
2. Specify the line number
3. Clearly describe the nature of the error

(b) Classification of Errors (Understanding + Analysis)

1. Classify each identified error into the appropriate compiler phase:
2. Lexical Errors
3. Syntax Errors
4. Semantic Errors
5. Logical Errors
6. Runtime Errors

(c) Compilation Behavior (Evaluation Level)

1. Analyze which errors will be detected during compilation and which will only appear during execution.
2. Explain how early-stage errors (lexical/syntax) may prevent detection of later-stage errors.

(d) Code Correction (Application Level)

1. Rewrite the corrected version of the program such that:
2. All errors are removed
3. The program executes correctly
4. Proper syntax and semantics are maintained

(e) Impact Analysis (Evaluation Level)

1. Discuss the impact of at least three errors on program execution.
2. Explain how incorrect constructs like improper pointer usage, division by zero, and infinite loops affect system behavior.

(f) Advanced Compiler Perspective (Create Level)

1. Suggest how a compiler can be improved to:
2. Provide better error messages
3. Perform early detection of runtime errors
4. Propose at least two enhancements in compiler design for efficient debugging.

Answer:

Description:

This question asks us to find the errors in a C program, identify their types, understand when they occur, correct the program, and explain their effect on execution.

(a) Error Identification

Line	Error	Description
6	i <= n	Logical error; should be i < n.
13	Missing ;	Syntax error.
15	sumArray(arr)	Semantic error; second argument is missing.
18	Missing ;	Syntax error.
22	a / b, b = 0	Runtime error; division by zero.
24	*p = 20	Runtime error; pointer has no valid address.

Lexical error: No lexical error is present.

(b) Classification

- **Lexical:** None
- **Syntax:** Lines 13, 18
- **Semantic:** Line 15
- **Logical:** Line 6
- **Runtime:** Lines 22, 24

(c) Compilation Behaviour

Syntax and semantic errors are found during compilation. Division by zero and invalid pointer access can occur during execution. Early syntax errors may stop the compiler from properly checking later statements.

(d) Correct Code

```
#include <stdio.h>

int sumArray(int arr[], int n)
{
   int i, sum = 0;

   for(i = 0; i < n; i++)
      sum += arr[i];

   return sum;
}

int main()
{
   int arr[5] = {1,2,3,4,5};
   int total = sumArray(arr, 5);

   if(total == 15)
      printf("Correct Sum\n");

   int a = 10, b = 2;
   int c = a / b;

   int value = 20;
   int *p = &value;

   printf("Division = %d\n", c);
   printf("Pointer = %d\n", *p);

   return 0;
}
```

Output

```
Correct Sum
Division = 5
Pointer = 20
```

(e) Impact Analysis

- **Pointer error:** May access invalid memory and crash the program.
- **Division by zero:** Causes invalid arithmetic during execution.
- **Array error:** May access memory outside the array.
- **Infinite loop:** Can keep the program running continuously and use CPU resources.

(f) Compiler Improvements

1. Give clear error messages with the **line number and correction**.
2. Use **static analysis** to detect possible runtime problems early.
3. Provide better debugging information such as variable values and error locations.

SCREENSHORT :

Main.cpp

```c
#include <stdio.h>

int sumArray(int arr[], int n)
{
    int i, sum = 0;

    for(i = 0; i < n; i++)
        sum += arr[i];

    return sum;
}

int main()
{
    int arr[5] = {1,2,3,4,5};
    int total = sumArray(arr, 5);

    if(total == 15)
        printf("Correct Sum\n");

    int a = 10, b = 2;
    int c = a / b;

    int value = 20;
    int *p = &value;

    printf("Division = %d\n", c);
    printf("Pointer = %d\n", *p);

    return 0;
}
```

OUTPUT:

```
Correct Sum
Division = 5
Pointer = 20
```

Q2.The following C program contains multiple lexical errors introduced intentionally. These errors occur due to invalid tokens, incorrect identifiers, malformed constants, and illegal symbols, which are typically detected during the lexical analysis phase of a compiler.Carefully examine the program and answer the questions given below.

```
#include <stdio.h>
int main() {
   int 1num = 100;
   float rate@ = 25.5;
   char ch = 'abc';
   int total = 50$20;
   int num#1 = 30;
   int %count = 40;

   printf("Welcome to Compiler Design);

   int x = 09;
   int y = 0xZ12;
   int z = 12.3.4;

   char str1[] = "Hello;
   char str2[] = "World";

   int _valid = 10;
   int invalid-id = 20;

   float value = 5..6;
   int data = 10@5;

   char c = 'xy';
   int 3value = 60;

   printf("Value: %d\n", total);

   return 0;
}
```

Tasks

(a) Error Identification

i) Identify all lexical errors in the program.

ii) List the line number and the invalid token.

(b) Error Classification

i) For each error, explain why it is considered a lexical error.

ii) Specify which rule of token formation is violated:

iii) Identifier rules

iv) Constant formation

vi) Operator/symbol rules

(c) Token Correction

i)Rewrite each invalid token into a valid token.

ii)Example: 2value → value2

(d) Compiler Behavior Analysis

i) Explain how a lexical analyzer (scanner) processes such errors.

ii)Discuss whether the compiler:

iii)Stops immediately

iv)Continues scanning using error recovery

(e) Advanced Thinking

i) Design a simple rule (or DFA idea) to detect invalid identifiers starting with digits.

ii) Suggest how error messages can be improved for beginners.

ANSWER:

Description

This question asks us to find **lexical errors** in a C program. We need to identify invalid tokens, explain why they are wrong, give their corrections, and understand how a lexical analyzer handles them.

(a) Error Identification

Line	Invalid Token	Error
4	1num	Identifier starts with a digit
5	rate@	@ is not allowed in an identifier
6	'abc'	Character literal contains multiple characters
7	50$20	$ is an invalid symbol
8	num#1	# is not allowed in an identifier
9	%count	% cannot start an identifier
10	"Welcome to Compiler Design)	Closing " is missing
11	09	Invalid octal number
12	0xZ12	Z is invalid in hexadecimal number
13	12.3.4	Invalid floating-point constant
14	"Hello	Closing " is missing
16	invalid-id	- cannot be part of an identifier
17	5..6	Invalid floating-point constant
18	10@5	@ is an invalid symbol
19	'xy'	Character literal contains multiple characters
20	3value	Identifier starts with a digit

(b) Error Classification

Identifier Rules

Identifiers can contain **letters, digits and _**, but cannot start with a digit.

Examples:

1num → Wrong
3value → Wrong

These constants are invalid:

09
0xZ12
12.3.4
5..6

They do not follow valid C number formats.

String and Character Rules

Wrong:

'abc'
'xy'
"Hello

A character literal should contain one character, and a string must have closing quotes.

Operator/Symbol Rules

These contain invalid symbols:

50$20
num#1
%count
10@5

(c) Token Correction

Wrong Token	Correct Token
1num	num1
rate@	rate
'abc'	'a'
50$20	50+20
num#1	num1
%count	count
"Welcome to Compiler Design)	"Welcome to Compiler Design"
09	9
0xZ12	0x12
12.3.4	12.34
"Hello	"Hello"
invalid-id	invalid_id
5..6	5.6
10@5	10+5
'xy'	'x'
3value	value3

(d) Compiler Behavior

The **lexical analyzer** reads the program character by character and groups them into tokens.

When it finds an invalid token, it reports a lexical error.

A compiler may either:

- **Stop** after a serious lexical error, or
- **Continue scanning** using error recovery and report more errors.

Continuing the scan is useful because the programmer can fix several errors at once.

SCREENSHORT :

Main.cpp

```c
#include <stdio.h>

int main()
{
    int num1 = 100;
    float rate = 25.5;
    char ch = 'a';
    int total = 50 + 20;
    int num1a = 30;
    int count = 40;

    printf("Welcome to Compiler Design");

    int x = 9;
    int y = 0x12;
    int z = 12.34;

    char str1[] = "Hello";
    char str2[] = "World";

    int _valid = 10;
    int invalid_id = 20;
    float value = 5.6;
    int data = 10 + 5;
    char c = 'x';
    char c = 'x';
    int value3 = 60;

    printf("\nValue: %d\n", total);

    return 0;
}
```

OUTPUT:

```
Welcome to Compiler Design
Value: 70
```

Q3. The following C program is designed to perform arithmetic operations, function calls, conditional checks, pointer usage, and looping. However, the program contains multiple errors distributed across different phases of a compiler, including:

A. Lexical Errors
B. Syntax Errors
C. Semantic Errors
D. Intermediate Code Generation Issues (logic/precedence)
E. Code Optimization Issues (redundancy/inefficiency)
F. Runtime / Target Code Errors

Carefully analyze the program and answer the questions below.

```
#include <stdio.h>

int divide(int a, int b) {
   return a / b
}

int main() {
   int #value = 20;
   float num = 15.5
   int result;

   result = divide(10);

   if(result = 5) {
      printf("Result is 5\n")
   }

   int x = 10;
   int y = 0;
   int z = x / y;

   int arr[4] = {1,2,3,4};
   printf("%d\n", arr[4]);

   int *ptr;
   *ptr = 25;


   return 0;
}
```

Do the following Tasks

(a) Identify Errors (Analysis Level)

I. Identify **all errors** in the program.

II. Mention **line number + error description**.

(b) Classify by Compiler Phase

Classify each error into:

I. Lexical (invalid symbols like @num)

II. Syntax (missing semicolon, brackets)

III. Semantic (function argument mismatch)

IV. Intermediate Code Issues (wrong expression handling/precedence)

V. Optimization Issues (redundant + 0)

VI. Runtime Errors (division by zero, invalid pointer access)

(c) Compilation vs Execution (Evaluation)

I. Which errors are detected at **compile time**?

II. Which errors appear only during **execution**?

III. Explain why some errors prevent further compilation.

(d) Correct the Program (Application)

I. Rewrite the **fully corrected program**.

II. Ensure:

   A. Proper syntax
   B. Correct semantics
   C. Safe execution

(e) Deep Analysis (Evaluation)

I. Explain how **incorrect pointer usage** affects runtime.

II. Analyze how **array bounds violation** impacts memory.

III. Discuss how **operator precedence errors** affect intermediate code.

(f) Advanced Task (Create Level)

I. Suggest **2 improvements** in compiler design to:

   A. Detect runtime errors earlier
   B. Improve debugging messages

II. Propose how **optimization phase** can remove redundant expressions.

ANSWER:

Description

The given C program contains different errors such as lexical, syntax, semantic, optimization, and runtime errors. The task is to find and correct these errors.

(a) Identify Errors

Line	Error	Type
4	Missing ;	Syntax
5	Missing }	Syntax
9	#value is invalid	Lexical
10	Missing ;	Syntax
12	Function needs 2 arguments	Semantic
13	= should be ==	Logic
14	Missing ;	Syntax
19	Division by zero	Runtime
22	arr[4] is outside the array	Runtime
25	Pointer is not initialized	Runtime
28	+ 0 is unnecessary	Optimization
30	Infinite loop	Runtime

(b) Classification

- **Lexical:** #value
- **Syntax:** Missing ;, }
- **Semantic:** Wrong function arguments
- **Intermediate/Logic:** result = 5 instead of result == 5
- **Optimization:** Unnecessary + 0
- **Runtime:** Division by zero, array error, pointer error, infinite loop

(c) Compilation vs Execution

Compile-time: Lexical, syntax, and semantic errors are mainly found during compilation.

Execution-time: Division by zero, invalid pointer, array out-of-bounds, and infinite loop occur during execution.

Syntax errors can stop compilation because the compiler cannot understand the program structure.

(d) Correct Program

```
#include <stdio.h>

int divide(int a, int b)
{
   return a / b;
}

int main()
{
   int value = 20;
   float num = 15.5;
   int result = divide(10, 2);

   if(result == 5)
      printf("Result is 5\n");

   int x = 10;
   int y = 2;
   int z = x / y;

   int arr[4] = {1, 2, 3, 4};
   printf("%d\n", arr[3]);

   int value2 = 25;
   int *ptr = &value2;
   *ptr = 25;

   int total = x + y * result;

   while(x < 20)
   {
      printf("%d\n", x);
      x++;
   }

   return 0;
}
```

Output

```
Result is 5
4
10
11
12
13
14
15
16
```

(e) Deep Analysis

Pointer: An uninitialized pointer may access an invalid memory location and crash the program.

Array: arr[4] is invalid because the valid indexes are 0 to 3. It may access unwanted memory.

Operator Precedence: In:

x + y * result

* is performed first, so it becomes:

x + (y * result)

(f) Improvements

1. **Early error detection:** Compiler can warn about division by zero, invalid pointers, and array bounds.
2. **Better error messages:** Show the line number and explain how to correct the error.

Optimization: The compiler can remove unnecessary expressions like:

x + y * result + 0

and convert them to:

x + y * result

SCREENSHORT:

Main.cpp

```c
#include <stdio.h>

int divide(int a, int b)
{
    return a / b;
}

int main()
{
    int value = 20;
    float num = 15.5;
    int result = divide(10, 2);

    if(result == 5)
        printf("Result is 5\n");

    int x = 10;
    int y = 2;
    int z = x / y;

    int arr[4] = {1, 2, 3, 4};
    printf("%d\n", arr[3]);

    int value2 = 25;
    int *ptr = &value2;
    *ptr = 25;

    int total = x + y * result;

    while(x < 20)
    {
        printf("%d\n", x);
        x++;
    }

    return 0;
}
```

OUTPUT:

```
Result is 5
4
10
11
12
13
14
15
16
17
18
19
```

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand